EDUTECH AI PLATFORM · SERVICE RECORD

● COMPLETE · PASS

Psychological Profiling Service

psych-svc

Big Five · RIASEC · Career Mapping

STATUS	TESTS	BUILD	GENERATED
Frozen	PASS Passing	SUCCESS	March 07, 2026

psych-svc — COMPLETION DOCUMENT

STATUS: FROZEN — IMMUTABLE AFTER 2026-03-07
This document is a permanent, authoritative record of the psych-svc microservice as delivered and verified. It must never be edited after creation.

0. Vital Statistics

ATTRIBUTE	VALUE
Service name	psych-svc
Maven artifact	`com.edutech:psych-svc:1.0.0-SNAPSHOT`
Root package	`com.edutech.psych`
DB schema	`psych_schema`
Flyway migrations	V1–V5 (5 scripts, with seeded reference data in V2)
Source files written	~70 (Domain+App: 42, Infra: 13, API+Bootstrap+Tests: ~15)
Test results	**12/12 PASS** — BUILD SUCCESS in 7.266 s
ArchUnit rules	5/5 PASS
Service tests	7/7 PASS (SessionServiceTest)
Build method	3 parallel specialist agents
Spring Boot version	3.x (Jakarta EE 10, Hibernate 6.x, virtual threads)
Java version	17
Architecture pattern	Hexagonal (Ports & Adapters), strict layer isolation
AI integration	WebClient → Python FastAPI sidecar (`psych-ai-svc`)
Resilience	Circuit breaker on psych-ai-svc (Resilience4j)

1. Bounded Context

psych-svc owns the **Psychometric Profiling** bounded context within the EduTech AI Platform. It is the single source of truth for:

- Student psychometric profiles — Big Five personality traits (Openness, Conscientiousness, Extraversion, Agreeableness, Neuroticism) + RIASEC career type code
- Reference trait dimension master data (seeded, read-only)
- Assessment session lifecycle (SCHEDULED → IN_PROGRESS → COMPLETED / CANCELLED)
- Career mapping recommendations — requested in-service, generated via Python AI sidecar
- Vector embedding storage placeholder (`embeddingJson` as TEXT — upgrade path to pgvector documented)

psych-svc does **not** own student identity, center/batch metadata, fee data, or academic assessment. It references entities from other services by UUID only and is fully autonomous given only the JWT token.

---

2. Architecture Philosophy

Hexagonal Architecture — Enforced by ArchUnit

```
┌──────────────────────────────────────────────────────────┐
│  api (REST controllers, global exception handler)        │
├──────────────────────────────────────────────────────────┤
│  application (services, DTOs, exceptions, config ports)  │
├──────────────────────────────────────────────────────────┤
│  domain (entities, enums, events, ports IN/OUT)          │
├──────────────────────────────────────────────────────────┤
│  infrastructure (persistence, security, messaging,       │
│                  external AI sidecar)                    │
└──────────────────────────────────────────────────────────┘
```

Dependency rule (immutable):
- `domain` depends on NOTHING (zero Spring, zero Jakarta except JPA annotations)
- `application` depends on `domain` only (+ SLF4J + Jackson ObjectMapper for career serialization)
- `infrastructure` depends on `domain` and `application`
- `api` depends on `application` and `domain`
- `infrastructure` does NOT depend on `api`
- `api` does NOT depend on `infrastructure`

Five ArchUnit Rules (all PASS)

RULE	DESCRIPTION
1	`domain` classes must not depend on `infrastructure` or `api`
2	`application` classes must not depend on `infrastructure` or `api`
3	`infrastructure` classes must not depend on `api`
4	`api` classes must not depend on `infrastructure`
5	`.*Service` classes must reside in `application.service` (excludes `PsychAiSvcWebClientAdapter`)

Design Invariants

INVARIANT	IMPLEMENTATION
No Lombok	Manual constructors, no annotations
No hardcoded values	All config via `${ENV_VAR}` in `application.yml`
Soft delete	`deleted_at IS NULL` filter on all mutable-entity JPQL queries
Optimistic locking	`@Version Long version` on PsychProfile, SessionHistory, CareerMapping

INVARIANT	IMPLEMENTATION
Static factory pattern	`Entity.create(...)` sets `id = UUID.randomUUID()` pre-persist
UUID generation	`@GeneratedValue(strategy = GenerationType.UUID)` — Hibernate 6.x uses pre-set value
Best-effort Kafka	Publish failures logged, never roll back DB transaction
RFC 7807 errors	`ProblemDetail.forStatusAndDetail(status, detail)` with `https://edutech.com/problems/{type}` URIs
Constructor injection	All Spring beans use constructor injection exclusively
Virtual threads	`spring.threads.virtual.enabled: true`
WebClient + `.block()`	Safe under virtual threads for AI sidecar calls
Circuit breaker	`@CircuitBreaker(name = "psych-ai-svc")` on `PsychAiSvcWebClientAdapter.predictCareers()`

3. Complete Package Structure (~70 files)

```
com.edutech.psych
├── PsychSvcApplication.java                        @SpringBootApplication @ConfigurationPropertiesScan
│
├── domain/
│   ├── model/
│   │   ├── PsychProfile.java                       @Entity — core aggregate
│   │   ├── TraitDimension.java                     @Entity — reference data (no soft delete)
│   │   ├── SessionHistory.java                     @Entity — assessment session lifecycle
│   │   ├── CareerMapping.java                      @Entity — AI-generated career recommendations
│   │   ├── ProfileStatus.java                      DRAFT | ACTIVE | ARCHIVED
│   │   ├── SessionType.java                        INITIAL | PERIODIC | TRIGGERED
│   │   ├── SessionStatus.java                      SCHEDULED | IN_PROGRESS | COMPLETED | CANCELLED
│   │   ├── CareerMappingStatus.java                PENDING | GENERATED | FAILED
│   │   └── Role.java                               STUDENT | CENTER_ADMIN | SUPER_ADMIN
│   ├── event/
│   │   ├── PsychProfileCreatedEvent.java           record
│   │   ├── SessionCompletedEvent.java              record
│   │   └── CareerMappingGeneratedEvent.java        record
│   └── port/
│       ├── in/
│       │   ├── CreatePsychProfileUseCase.java
│       │   ├── StartSessionUseCase.java
│       │   ├── CompleteSessionUseCase.java
│       │   └── RequestCareerMappingUseCase.java
│       └── out/
│           ├── PsychProfileRepository.java
│           ├── TraitDimensionRepository.java
│           ├── SessionHistoryRepository.java
│           ├── CareerMappingRepository.java
│           ├── PsychEventPublisher.java
│           ├── PsychAiSvcClient.java               interface → Python sidecar
│           └── CareerPredictionResponse.java      record(topCareers, reasoning, modelVersion)
│
├── application/
│   ├── config/
│   │   ├── JwtProperties.java                      @ConfigurationProperties("jwt")
│   │   └── PsychAiSvcProperties.java               @ConfigurationProperties("psych-ai-svc")
│   ├── dto/
│   │   ├── AuthPrincipal.java                      record — userId,email,role,centerId,deviceFP
│   │   ├── CreatePsychProfileRequest.java          record + @NotNull on all fields
│   │   ├── PsychProfileResponse.java               record
│   │   ├── StartSessionRequest.java                record + @NotNull
```

```
│   │   ├── SessionResponse.java                    record
│   │   ├── CompleteSessionRequest.java             record + @DecimalMin/Max on trait scores
│   │   └── CareerMappingResponse.java              record
│   ├── exception/
│   │   ├── PsychException.java                     abstract base (RuntimeException)
│   │   ├── PsychProfileNotFoundException.java
│   │   ├── SessionHistoryNotFoundException.java
│   │   ├── CareerMappingNotFoundException.java
│   │   ├── PsychAccessDeniedException.java
│   │   ├── SessionAlreadyCompletedException.java
│   │   └── ProfileNotActiveException.java
│   └── service/
│       ├── PsychProfileService.java                implements CreatePsychProfileUseCase
│       ├── SessionService.java                     implements StartSessionUseCase, CompleteSessionUseCase
│       └── CareerMappingService.java               implements RequestCareerMappingUseCase
│
├── infrastructure/
│   ├── config/
│   │   └── KafkaTopicProperties.java               @ConfigurationProperties("kafka.topics")
│   ├── security/
│   │   ├── JwtTokenValidator.java                  JJWT 0.12.x RS256
│   │   ├── JwtAuthenticationFilter.java            OncePerRequestFilter + MDC
│   │   └── SecurityConfig.java                     stateless, @EnableMethodSecurity
│   ├── persistence/
│   │   ├── SpringDataPsychProfileRepository.java   package-private JpaRepository
│   │   ├── PsychProfilePersistenceAdapter.java     @Component implements PsychProfileRepository
│   │   ├── SpringDataTraitDimensionRepository.java package-private
│   │   ├── TraitDimensionPersistenceAdapter.java   @Component implements TraitDimensionRepository
│   │   ├── SpringDataSessionHistoryRepository.java package-private
│   │   ├── SessionHistoryPersistenceAdapter.java   @Component implements SessionHistoryRepository
│   │   ├── SpringDataCareerMappingRepository.java  package-private
│   │   └── CareerMappingPersistenceAdapter.java    @Component implements CareerMappingRepository
│   ├── messaging/
│   │   ├── PsychEventKafkaAdapter.java             implements PsychEventPublisher
│   │   └── AssessEventConsumer.java                @KafkaListener(assess-events)
│   └── external/
│       └── PsychAiSvcWebClientAdapter.java         implements PsychAiSvcClient via WebClient
│
├── api/
│   ├── GlobalExceptionHandler.java                 @RestControllerAdvice RFC 7807
│   ├── PsychProfileController.java                 /api/v1/psych/profiles
│   ├── SessionController.java                      /api/v1/psych/profiles/{profileId}/sessions
│   └── CareerMappingController.java                /api/v1/psych/profiles/{profileId}/career-mappings
│
└── [test]
    ├── architecture/
    │   └── ArchitectureRulesTest.java              5 ArchUnit rules
    └── application/service/
        └── SessionServiceTest.java                 7 Mockito unit tests
```

4. Domain Model — Deep Specification

4.1 PsychProfile Entity — Core Aggregate

```
PsychProfile
├── id:              UUID        (PK, immutable)
├── studentId:       UUID        (immutable — cross-service ref)
├── centerId:        UUID        (immutable — cross-service ref)
├── batchId:         UUID        (immutable — cross-service ref)
├── openness:        double      (Big Five O — 0.0-1.0)
```

```
├── conscientiousness: double        (Big Five C — 0.0-1.0)
├── extraversion:      double        (Big Five E — 0.0-1.0)
├── agreeableness:     double        (Big Five A — 0.0-1.0)
├── neuroticism:       double        (Big Five N — 0.0-1.0)
├── riasecCode:        String        (top RIASEC letters e.g. "RIA", nullable)
├── embeddingJson:     String        (TEXT — pgvector placeholder, null until AI enrichment)
├── status:            ProfileStatus (DRAFT initial)
├── version:           Long          (@Version)
├── createdAt:         Instant       (immutable)
├── updatedAt:         Instant
└── deletedAt:         Instant       (soft delete)
```

State machine:

```
DRAFT ──activate()──→ ACTIVE ──archive()──→ ARCHIVED
       (guard: DRAFT)         (guard: ACTIVE)
```

Domain methods:
- `activate()` — DRAFT → ACTIVE (guard: must be DRAFT, else IllegalStateException)
- `archive()` — ACTIVE → ARCHIVED (guard: must be ACTIVE)
- `updateTraits(o,c,e,a,n,riasec)` — updates all 5 Big Five + RIASEC, guard: must not be ARCHIVED

Factory: `PsychProfile.create(studentId, centerId, batchId)` → all traits=0.0, riasecCode=null, status=DRAFT

DB constraint: `UNIQUE(student_id, center_id)` — one profile per student per center

4.2 TraitDimension Entity — Reference Data

```
TraitDimension
├── id:          UUID    (PK, immutable)
├── code:        String  (UNIQUE — e.g. "OPENNESS", "RIASEC_R")
├── name:        String  (e.g. "Openness to Experience")
├── category:    String  ("BIG_FIVE" | "RIASEC")
├── description: String  (TEXT)
└── createdAt:   Instant (immutable)
```

Special characteristics:
- No `@Version` (reference data — no optimistic lock needed)
- No soft delete (reference data is permanent)
- No domain mutation methods (read-only after seeding)
- No `updatedAt` field

Seeded in V2 migration (11 rows):
- Big Five: OPENNESS, CONSCIENTIOUSNESS, EXTRAVERSION, AGREEABLENESS, NEUROTICISM
- RIASEC: RIASEC_R (Realistic), RIASEC_I (Investigative), RIASEC_A (Artistic), RIASEC_S (Social), RIASEC_E (Enterprising), RIASEC_C (Conventional)

4.3 SessionHistory Entity

```
SessionHistory
├── id:          UUID          (PK, immutable)
├── profileId:   UUID          (immutable, FK → psych_profiles)
├── studentId:   UUID          (immutable)
├── centerId:    UUID          (immutable)
├── sessionType: SessionType   (immutable — INITIAL | PERIODIC | TRIGGERED)
├── scheduledAt: Instant       (immutable)
├── startedAt:   Instant       (set by start(), nullable)
├── completedAt: Instant       (set by complete(), nullable)
├── notes:       String        (TEXT, set by complete(), nullable)
├── status:      SessionStatus (SCHEDULED initial)
├── version:     Long          (@Version)
```

```
├── createdAt:   Instant       (immutable)
├── updatedAt:   Instant
└── deletedAt:   Instant       (soft delete)
```

State machine:

```
SCHEDULED ──start()──→ IN_PROGRESS ──complete(notes)──→ COMPLETED
    └──cancel()──→ CANCELLED     └──cancel()──→ CANCELLED
```

4.4 CareerMapping Entity

```
CareerMapping
├── id:           UUID                (PK, immutable)
├── profileId:    UUID                (immutable, FK → psych_profiles)
├── studentId:    UUID                (immutable)
├── centerId:     UUID                (immutable)
├── requestedAt:  Instant             (immutable — set at creation)
├── generatedAt:  Instant             (set by complete(), nullable)
├── topCareers:   String              (TEXT — JSON array, nullable)
├── reasoning:    String              (TEXT — AI reasoning, nullable)
├── modelVersion: String              (nullable — e.g. "psych-ai-v2")
├── status:       CareerMappingStatus (PENDING initial)
├── version:      Long                (@Version)
├── createdAt:    Instant             (immutable)
├── updatedAt:    Instant
└── deletedAt:    Instant             (soft delete)
```

State machine:

```
PENDING ──complete(topCareers, reasoning, modelVersion)──→ GENERATED
PENDING ──fail()──────────────────────────────────────────→ FAILED
```

topCareers serialization: Jackson `ObjectMapper` in `CareerMappingService` serializes `List<String>` → JSON string before calling `mapping.complete()`. This keeps the domain entity free of Jackson annotations.

---

5. Domain Events

All events are Java `record` types. Published best-effort — Kafka failures logged, never rolled back.

EVENT	PUBLISHER	PAYLOAD FIELDS
`PsychProfileCreatedEvent`	PsychProfileService.createProfile()	profileId, studentId, centerId, batchId
`SessionCompletedEvent`	SessionService.completeSession()	sessionId, profileId, studentId, centerId
`CareerMappingGeneratedEvent`	CareerMappingService.requestCareerMapping()	mappingId, profileId, studentId, centerId, topCareers

---

6. Ports Specification

6.1 Ports IN (Use Cases)

INTERFACE	METHOD	IMPLEMENTOR
`CreatePsychProfileUseCase`	`createProfile(CreatePsychProfileRequest, AuthPrincipal): PsychProfileResponse`	PsychProfileService

INTERFACE	METHOD	IMPLEMENTOR
`StartSessionUseCase`	`startSession(UUID profileId, StartSessionRequest, AuthPrincipal): SessionResponse`	SessionService
`CompleteSessionUseCase`	`completeSession(UUID sessionId, CompleteSessionRequest, AuthPrincipal): SessionResponse`	SessionService
`RequestCareerMappingUseCase`	`requestCareerMapping(UUID profileId, AuthPrincipal): CareerMappingResponse`	CareerMappingService

6.2 Ports OUT (Repository + Publisher + External)

INTERFACE	KEY METHODS
`PsychProfileRepository`	`findById`, `findByStudentIdAndCenterId`, `findByCenterId`, `save`
`TraitDimensionRepository`	`findAll`, `findByCode`
`SessionHistoryRepository`	`findById`, `findByProfileId`, `save`
`CareerMappingRepository`	`findById`, `findByProfileId`, `save`
`PsychEventPublisher`	`publish(Object event)`
`PsychAiSvcClient`	`predictCareers(profileId, o,c,e,a,n, riasecCode): CareerPredictionResponse`

7. Application Services — Execution Traces

7.1 PsychProfileService.createProfile

```
Input: CreatePsychProfileRequest(studentId, centerId, batchId), AuthPrincipal

1. Guard: principal.belongsToCenter(req.centerId()) → or PsychAccessDeniedException
2. PsychProfile.create(req.studentId(), req.centerId(), req.batchId())
3. profile.activate()  — immediately move DRAFT → ACTIVE
4. profileRepository.save(profile)
5. eventPublisher.publish(PsychProfileCreatedEvent)
6. Return toResponse(profile)
```

7.2 SessionService.startSession

```
Input: profileId, StartSessionRequest(sessionType, scheduledAt), AuthPrincipal

1. profile = profileRepository.findById(profileId) → or PsychProfileNotFoundException
2. Guard: principal.userId == profile.getStudentId() || principal.isSuperAdmin()
         → or PsychAccessDeniedException
3. Guard: profile.getStatus() == ProfileStatus.ACTIVE → or ProfileNotActiveException
4. SessionHistory.create(profileId, studentId, centerId, sessionType, scheduledAt)
5. session.start()  — immediately start: SCHEDULED → IN_PROGRESS
6. sessionHistoryRepository.save(session)
7. Return toResponse(session)
```

7.3 SessionService.completeSession — Full Trace

```
Input: sessionId, CompleteSessionRequest(o,c,e,a,n, riasecCode, notes), AuthPrincipal

@Transactional — all in one DB transaction:

1. session = sessionHistoryRepository.findById(sessionId) → or SessionHistoryNotFoundException
```

```
2. Guard: principal.userId == session.getStudentId() || principal.isSuperAdmin()
        → or PsychAccessDeniedException
3. Guard: session.getStatus() == SessionStatus.IN_PROGRESS
        → or SessionAlreadyCompletedException
4. session.complete(req.notes())  — IN_PROGRESS → COMPLETED, completedAt=now()
5. sessionHistoryRepository.save(session)
6. profile = profileRepository.findById(session.getProfileId()) → or PsychProfileNotFoundException
7. profile.updateTraits(req.openness(), req.conscientiousness(), req.extraversion(),
                        req.agreeableness(), req.neuroticism(), req.riasecCode())
8. profileRepository.save(profile)
9. eventPublisher.publish(SessionCompletedEvent)  — best-effort
10. Return toResponse(session)
```

7.4 CareerMappingService.requestCareerMapping

```
Input: profileId, AuthPrincipal

@Transactional:

1. profile = profileRepository.findById(profileId) → or PsychProfileNotFoundException
2. Guard: ownership or isSuperAdmin → or PsychAccessDeniedException
3. Guard: profile.getStatus() == ACTIVE → or ProfileNotActiveException
4. CareerMapping.create(profileId, studentId, centerId)
5. mapping = careerMappingRepository.save(mapping)  — PENDING status persisted

Try:
6.  prediction = psychAiSvcClient.predictCareers(
        profile.getId(), o, c, e, a, n, riasecCode)
    @CircuitBreaker on this call
7.  topCareersJson = objectMapper.writeValueAsString(prediction.topCareers())
8.  mapping.complete(topCareersJson, prediction.reasoning(), prediction.modelVersion())
         — PENDING → GENERATED, generatedAt=now()
9.  careerMappingRepository.save(mapping)
10. eventPublisher.publish(CareerMappingGeneratedEvent)

Catch Exception:
11. log.error("Career mapping failed for profileId={}: {}", profileId, e.getMessage())
12. mapping.fail()  — PENDING → FAILED
13. careerMappingRepository.save(mapping)

14. Return toResponse(mapping)
```

Key design: The AI call happens *within* the `@Transactional` boundary. If the AI succeeds → GENERATED saved atomically. If it fails → FAILED saved atomically. The DB always has a definitive terminal state for the mapping.

8. Authorization Matrix

ACTION	STUDENT	CENTER_ADMIN	SUPER_ADMIN
Create profile	✕	Own center only	✓
View profile	✓ Own only	Own center	✓
List profiles by center	✕	Own center	✓
Start session	✓ Own profile	✕	✓
Complete session	✓ Own session	✕	✓
View sessions	✓ Own profile	✕	✓

ACTION	STUDENT	CENTER_ADMIN	SUPER_ADMIN
Request career mapping	✓ Own profile	×	✓
View career mappings	✓ Own profile	×	✓

AuthPrincipal record fields: `userId`, `email`, `role`, `centerId` (nullable), `deviceFP`
Helper methods: `isSuperAdmin()`, `isStudent()`, `belongsToCenter(UUID centerId)`

9. API Contract

PsychProfileController — `/api/v1/psych/profiles`

METHOD	PATH	REQUEST	RESPONSE	AUTH
POST	`/api/v1/psych/profiles`	`CreatePsychProfileRequest`	`201 PsychProfileResponse`	CENTER_ADMIN / SUPER_ADMIN
GET	`/api/v1/psych/profiles/{profileId}`	—	`200 PsychProfileResponse`	STUDENT (own) / CENTER_ADMIN (own center) / SUPER_ADMIN
GET	`/api/v1/psych/profiles?centerId={uuid}`	—	`200 List<PsychProfileResponse>`	CENTER_ADMIN (own center) / SUPER_ADMIN

SessionController — `/api/v1/psych/profiles/{profileId}/sessions`

METHOD	PATH	REQUEST	RESPONSE	AUTH
POST	`/api/v1/psych/profiles/{profileId}/sessions`	`StartSessionRequest`	`201 SessionResponse`	STUDENT
POST	`/api/v1/psych/profiles/{profileId}/sessions/{sessionId}/complete`	`CompleteSessionRequest`	`200 SessionResponse`	STUDENT (own)
GET	`/api/v1/psych/profiles/{profileId}/sessions`	—	`200 List<SessionResponse>`	STUDENT (own)

CareerMappingController — `/api/v1/psych/profiles/{profileId}/career-mappings`

METHOD	PATH	REQUEST	RESPONSE	AUTH
POST	`/api/v1/psych/profiles/{profileId}/career-mappings`	—	`201 CareerMappingResponse`	STUDENT (own)
GET	`/api/v1/psych/profiles/{profileId}/career-mappings`	—	`200 List<CareerMappingResponse>`	STUDENT (own)

Error Contract (RFC 7807)

All errors return `application/problem+json`:

```
{
  "type": "https://edutech.com/problems/psych-profile-not-found",
  "title": "Psych Profile Not Found",
  "status": 404,
  "detail": "Psych profile not found: 550e8400-e29b-41d4-a716-446655440000"
}
```

EXCEPTION	HTTP	PROBLEM TYPE
`PsychProfileNotFoundException`	404	`psych-profile-not-found`

EXCEPTION	HTTP	PROBLEM TYPE
`SessionHistoryNotFoundException`	404	`session-not-found`
`CareerMappingNotFoundException`	404	`career-mapping-not-found`
`PsychAccessDeniedException`	403	`access-denied`
`SessionAlreadyCompletedException`	409	`session-already-completed`
`ProfileNotActiveException`	409	`profile-not-active`
`MethodArgumentNotValidException`	400	`validation-error`
`Exception` (catch-all)	500	`internal-error`

10. Database Schema

Schema: `psych_schema` (PostgreSQL 15+)

V1 — Init Schema

```
CREATE SCHEMA IF NOT EXISTS psych_schema;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
-- CREATE EXTENSION IF NOT EXISTS vector;  -- Deferred: enable when pgvector installed
SET search_path TO psych_schema;
```

V2 — `trait_dimensions` (Reference Data with Seeds)

```
CREATE TABLE psych_schema.trait_dimensions (
    id          UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    code        TEXT        NOT NULL UNIQUE,
    name        TEXT        NOT NULL,
    category    TEXT        NOT NULL,
    description TEXT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_td_category CHECK (category IN ('BIG_FIVE', 'RIASEC'))
);

-- 11 seed rows inserted:
-- Big Five: OPENNESS, CONSCIENTIOUSNESS, EXTRAVERSION, AGREEABLENESS, NEUROTICISM
-- RIASEC: RIASEC_R, RIASEC_I, RIASEC_A, RIASEC_S, RIASEC_E, RIASEC_C
```

V3 — `psych_profiles`

```
CREATE TABLE psych_schema.psych_profiles (
    id                UUID             PRIMARY KEY DEFAULT gen_random_uuid(),
    student_id        UUID             NOT NULL,
    center_id         UUID             NOT NULL,
    batch_id          UUID             NOT NULL,
    openness          DOUBLE PRECISION NOT NULL DEFAULT 0.0 CHECK (openness BETWEEN 0.0 AND 1.0),
    conscientiousness DOUBLE PRECISION NOT NULL DEFAULT 0.0 CHECK (conscientiousness BETWEEN 0.0 AND 1.0),
    extraversion      DOUBLE PRECISION NOT NULL DEFAULT 0.0 CHECK (extraversion BETWEEN 0.0 AND 1.0),
    agreeableness     DOUBLE PRECISION NOT NULL DEFAULT 0.0 CHECK (agreeableness BETWEEN 0.0 AND 1.0),
    neuroticism       DOUBLE PRECISION NOT NULL DEFAULT 0.0 CHECK (neuroticism BETWEEN 0.0 AND 1.0),
    riasec_code       TEXT,
    embedding_json    TEXT,   -- placeholder; upgrade: ALTER COLUMN to vector(1536)
    status            TEXT             NOT NULL DEFAULT 'DRAFT',
    version           BIGINT           NOT NULL DEFAULT 0,
```

```
    created_at        TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at        TIMESTAMPTZ      NOT NULL DEFAULT now(),
    deleted_at        TIMESTAMPTZ,
    CONSTRAINT chk_pp_status CHECK (status IN ('DRAFT', 'ACTIVE', 'ARCHIVED')),
    CONSTRAINT uq_psych_profile_student_center UNIQUE (student_id, center_id)
);
-- Partial indexes: center_id, student_id, status WHERE deleted_at IS NULL
-- Trigger: trg_psych_profiles_updated_at → set_updated_at()
```

V4 — `session_histories`

```
CREATE TABLE psych_schema.session_histories (
    id           UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id   UUID        NOT NULL REFERENCES psych_schema.psych_profiles(id),
    student_id   UUID        NOT NULL,
    center_id    UUID        NOT NULL,
    session_type TEXT        NOT NULL,
    scheduled_at TIMESTAMPTZ NOT NULL,
    started_at   TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    notes        TEXT,
    status       TEXT        NOT NULL DEFAULT 'SCHEDULED',
    version      BIGINT      NOT NULL DEFAULT 0,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at   TIMESTAMPTZ,
    CONSTRAINT chk_sh_session_type CHECK (session_type IN ('INITIAL', 'PERIODIC', 'TRIGGERED')),
    CONSTRAINT chk_sh_status CHECK (status IN ('SCHEDULED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED'))
);
-- Partial indexes: profile_id, student_id, status WHERE deleted_at IS NULL
-- BRIN index: created_at (append-heavy table)
-- Trigger: trg_session_histories_updated_at
```

V5 — `career_mappings`

```
CREATE TABLE psych_schema.career_mappings (
    id            UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id    UUID        NOT NULL REFERENCES psych_schema.psych_profiles(id),
    student_id    UUID        NOT NULL,
    center_id     UUID        NOT NULL,
    requested_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    generated_at  TIMESTAMPTZ,
    top_careers   TEXT,   -- JSON array
    reasoning     TEXT,
    model_version TEXT,
    status        TEXT        NOT NULL DEFAULT 'PENDING',
    version       BIGINT      NOT NULL DEFAULT 0,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at    TIMESTAMPTZ,
    CONSTRAINT chk_cm_status CHECK (status IN ('PENDING', 'GENERATED', 'FAILED'))
);
-- Partial indexes: profile_id, student_id, status WHERE deleted_at IS NULL
-- BRIN index: requested_at (append-heavy table)
-- Trigger: trg_career_mappings_updated_at
```

Index Strategy Summary

TABLE	INDEX TYPE	PURPOSE
psych_profiles	B-tree partial `(center_id)` WHERE `deleted_at IS NULL`	Center profile queries
psych_profiles	B-tree partial `(student_id)` WHERE `deleted_at IS NULL`	Student profile lookup
psych_profiles	B-tree partial `(status)` WHERE `deleted_at IS NULL`	Status filtering
psych_profiles	Unique `(student_id, center_id)`	One profile per student per center
session_histories	B-tree partial `(profile_id)` WHERE `deleted_at IS NULL`	Profile session history
session_histories	BRIN `(created_at)`	Append-only access
career_mappings	B-tree partial `(profile_id)` WHERE `deleted_at IS NULL`	Profile career list
career_mappings	BRIN `(requested_at)`	Append-only access

11. Infrastructure Layer

11.1 JWT Security (RS256)

```
// JwtTokenValidator — JJWT 0.12.x
Jwts.parser()
    .verifyWith(publicKey)          // RSA public key from PEM file
    .requireIssuer(issuer)
    .build()
    .parseSignedClaims(token)
    .getPayload()                   // Claims → AuthPrincipal record
```

11.2 Persistence Adapters — Pattern

All persistence adapters:
1. Implement the domain `*Repository` interface (`@Component`, public)
2. Delegate to a **package-private** `SpringData*Repository extends JpaRepository`
3. Apply soft-delete filter: `deletedAtIsNull` in all derived queries
4. Never expose Spring Data interfaces outside `infrastructure.persistence`

TraitDimension exception: No soft delete filter — `findAll()` returns all reference data (trait dimensions are permanent master data).

Post-agent fix applied: Infrastructure agent initially wrote `delete()` methods that called `entity.setDeletedAt()` (no setter exists on domain entities). These were removed from all 4 adapters since `delete()` was not declared in any domain port interface.

11.3 Python AI Sidecar Integration

```
PsychAiSvcWebClientAdapter (implements PsychAiSvcClient)
└── WebClient baseUrl = ${PSYCH_AI_SVC_BASE_URL}
    POST /api/v1/predict-careers
    Request: PredictRequest record (profileId, o, c, e, a, n, riasecCode)
    Response: PredictResponse record → mapped to CareerPredictionResponse
    .block() — synchronous; safe under virtual threads
    @CircuitBreaker(name = "psych-ai-svc") — opens after threshold failures
    On exception: throws RuntimeException("Career prediction failed")
                  (caught by CareerMappingService → sets FAILED status)
```

11.4 Kafka Configuration

PsychEventKafkaAdapter:
- Routes `PsychProfileCreatedEvent`, `SessionCompletedEvent`, `CareerMappingGeneratedEvent` → `psychEvents` topic
- All events also published to `auditImmutable` topic
- Key = entity ID string for ordered processing
- Exceptions logged and swallowed (best-effort semantics)

AssessEventConsumer:

- Listens on `${kafka.topics.assess-events}` with per-listener `StringDeserializer` override
- Phase 1: logs received payload
- Phase 2 hook: trigger periodic session when assess-svc publishes poor grade events

---

12. Configuration Reference — All Environment Variables

Core

VARIABLE	DESCRIPTION
`PSYCH_SVC_NAME`	Service name for metrics/tracing
`PSYCH_SVC_PORT`	HTTP listen port

Database

VARIABLE	DESCRIPTION
`POSTGRES_HOST`	PostgreSQL hostname
`POSTGRES_PORT`	PostgreSQL port
`PSYCH_SVC_DB_NAME`	Database name
`PSYCH_SVC_DB_USERNAME`	DB user
`PSYCH_SVC_DB_PASSWORD`	DB password
`PSYCH_SVC_DB_POOL_MAX_SIZE`	HikariCP max pool size
`PSYCH_SVC_DB_POOL_MIN_IDLE`	HikariCP min idle
`PSYCH_SVC_DB_CONNECTION_TIMEOUT_MS`	Connection timeout
`PSYCH_SVC_DB_IDLE_TIMEOUT_MS`	Idle timeout

Redis, Kafka

VARIABLE	DESCRIPTION
`REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_SSL_ENABLED`	Redis connection
`KAFKA_BOOTSTRAP_SERVERS`	Kafka brokers
`PSYCH_SVC_KAFKA_CONSUMER_GROUP`	Consumer group ID
`KAFKA_TOPIC_PSYCH_EVENTS`	Outbound events topic
`KAFKA_TOPIC_ASSESS_EVENTS`	Inbound (from assess-svc) topic
`KAFKA_TOPIC_AUDIT_IMMUTABLE`	Audit log topic

JWT

VARIABLE	DESCRIPTION
`JWT_PUBLIC_KEY_PATH`	Path to RSA public key PEM file
`JWT_ISSUER`	Expected issuer claim in JWT

Python AI Sidecar

VARIABLE	DESCRIPTION
`PSYCH_AI_SVC_BASE_URL`	Base URL of psych-ai-svc (FastAPI)
`PSYCH_AI_SVC_CONNECT_TIMEOUT_SECONDS`	WebClient connect timeout
`PSYCH_AI_SVC_READ_TIMEOUT_SECONDS`	WebClient read timeout

pgvector / Embeddings

VARIABLE	DESCRIPTION
`AI_EMBEDDING_DIMENSIONS`	Embedding vector dimensions (e.g. 1536)
`AI_GATEWAY_BASE_URL`	Base URL of ai-gateway-svc

Resilience4j

VARIABLE	DESCRIPTION
`R4J_CB_AI_WINDOW_SIZE`	Circuit breaker sliding window size
`R4J_CB_AI_FAILURE_THRESHOLD`	Failure rate threshold (%)
`R4J_CB_AI_WAIT_DURATION`	Open-state wait before half-open

Observability

VARIABLE	DESCRIPTION
`ACTUATOR_ENDPOINTS`	Exposed actuator endpoints
`APP_ENVIRONMENT`	Environment tag for metrics
`OTEL_SAMPLING_PROBABILITY`	OpenTelemetry trace sampling rate
`OTEL_EXPORTER_OTLP_ENDPOINT`	OTLP collector endpoint
`LOG_LEVEL_ROOT`	Root logger level
`LOG_LEVEL_APP`	`com.edutech` logger level

13. Test Coverage

Test Run Results — 2026-03-07 — 12/12 PASS — BUILD SUCCESS — 7.266 s

ArchitectureRulesTest — 5/5 PASS

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.690 s
-- in com.edutech.psych.architecture.ArchitectureRulesTest
```

RULE	RESULT
domain must not depend on infrastructure or api	PASS
application must not depend on infrastructure or api	PASS
infrastructure must not depend on api	PASS
api must not depend on infrastructure	PASS
services must reside in application.service	PASS

SessionServiceTest — 7/7 PASS

```
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.206 s
-- in com.edutech.psych.application.service.SessionServiceTest
```

TEST	SCENARIO	VERIFIED
`startSession_success`	ACTIVE profile, owned student → IN_PROGRESS session returned	status=IN_PROGRESS
`startSession_profileNotFound`	Unknown profileId → `PsychProfileNotFoundException`	Exception type, sessionRepo never saves
`startSession_accessDenied`	Different userId → `PsychAccessDeniedException`	Exception type, sessionRepo never saves
`startSession_profileNotActive`	DRAFT profile → `ProfileNotActiveException`	Exception type, sessionRepo never saves
`completeSession_success`	Real domain transitions: `create()→start()`, then `completeSession()`	status=COMPLETED, eventPublisher called
`completeSession_notFound`	Unknown sessionId → `SessionHistoryNotFoundException`	Exception type
`completeSession_alreadyCompleted`	Session in COMPLETED state → `SessionAlreadyCompletedException`	Exception type

Test infrastructure:

- `@MockitoSettings(strictness = Strictness.LENIENT)` — prevents UnnecessaryStubbing for shared mock helpers
- Real `SessionHistory.create() → start() → complete()` used for state-transition tests
- All mock helpers extracted to local variables before use in `thenReturn()` (prevents Mockito UnfinishedStubbing)

14. Kafka Event Flow

```
psych-svc PUBLISHES (psych-events topic):
┌──────────────────────────────────────────┐
│ PsychProfileCreatedEvent  → profile lifecycle  │
│ SessionCompletedEvent     → after assessment   │
│ CareerMappingGeneratedEvent → after AI call    │
└──────────────────────────────────────────┘

psych-svc CONSUMES (assess-events topic):
┌──────────────────────────────────────────┐
│ AssessEventConsumer                            │
│ @KafkaListener(assess-events)                  │
│ Phase 1: log and discard                       │
│ Phase 2: trigger TRIGGERED session on low grade│
└──────────────────────────────────────────┘
```

15. Failure Modes and Resilience

FAILURE	BEHAVIOR
DB down	HTTP 503 — Spring propagates DataAccessException → GlobalExceptionHandler
psych-ai-svc down	Circuit breaker opens → CareerMappingService catches RuntimeException → mapping saved as FAILED, HTTP 201 returned with FAILED status
psych-ai-svc returns error	Same path — mapping → FAILED

FAILURE	BEHAVIOR
Kafka publish failure	Logged at WARN, DB transaction already committed, event silently dropped
Concurrent profile update	`@Version` → OptimisticLockingFailureException → 409 Conflict
Duplicate profile (same student+center)	DB UNIQUE constraint → DataIntegrityViolationException → 409
Redis down	Startup may fail if eagerly configured — no runtime caching layer in psych-svc (Redis present via pom but not used for caching in Phase 1)

16. Cross-Service Dependencies

```
psych-svc RECEIVES (by UUID reference):
  ← center-svc:  centerId, batchId
  ← user-svc:    studentId (via JWT sub claim)

psych-svc CONSUMES (Kafka):
  ← assess-svc: grade events (Phase 2: trigger psychological review on distress patterns)

psych-svc CALLS (HTTP — sync via WebClient):
  → psych-ai-svc: Python FastAPI sidecar for career prediction (POST /api/v1/predict-careers)

psych-svc PUBLISHES (Kafka):
  → audit pipeline: all events via audit-immutable topic
  → (future) notify-svc: CareerMappingGeneratedEvent (career report notification)
```

17. pgvector — Architecture Decision

`embeddingJson` is stored as `TEXT` (serialized `float[]` as JSON) rather than using the native `vector` type. Same decision as assess-svc:

Upgrade path:

```
-- Phase 2 upgrade (V6 migration):
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE psych_schema.psych_profiles ADD COLUMN embedding vector(1536);
UPDATE psych_schema.psych_profiles SET embedding = embedding_json::vector WHERE embedding_json IS NOT NULL;
ALTER TABLE psych_schema.psych_profiles DROP COLUMN embedding_json;
CREATE INDEX ON psych_schema.psych_profiles USING hnsw(embedding vector_cosine_ops);
```

18. Known Constraints and Upgrade Paths

#	CONSTRAINT	UPGRADE PATH
1	`embeddingJson` stored as TEXT	V6 migration: enable pgvector, `ALTER COLUMN`, HNSW index
2	Career mapping is synchronous (blocking WebClient)	Phase 2: make asynchronous via message queue + webhook pattern
3	Session only tracks one set of trait scores	Phase 2: add `SessionTraitScore` child entity per dimension for granular history
4	No rate limiting on career mapping requests	Phase 2: Redis-based rate limiter per studentId
5	psych-ai-svc integration is HTTP sync	Phase 2: async job queue (RabbitMQ / Kafka saga)
6	`TraitDimension` has no soft delete	By design: reference data is permanent; administrative changes require DB migration

19. Construction Record

AGENT	RESPONSIBILITY	FILES
Agent 1 — Domain + Application	5 enums, 4 entities, 3 events, 4 ports IN, 6 ports OUT, 2 config, 7 DTOs, 6 exceptions, 3 services	42 files
Agent 2 — Infrastructure	KafkaTopicProperties, JWT security (3), 8 persistence (4 Spring Data + 4 adapters), PsychEventKafkaAdapter, AssessEventConsumer, PsychAiSvcWebClientAdapter	13 files
Agent 3 — API + Bootstrap + Migrations + Tests	PsychSvcApplication, GlobalExceptionHandler, 3 controllers, 5 SQL migrations (V1-V5), application.yml kafka.topics edit, ArchitectureRulesTest, SessionServiceTest	~15 files
Total		**~70 files**

Post-agent fix applied:
Infrastructure agent added `delete()` methods to 3 persistence adapters that called `entity.setDeletedAt()`. Since:
1. Domain entities have no `setDeletedAt()` setter (no public setters by design)
2. `delete()` was not declared in any domain port interface (`PsychProfileRepository`, `SessionHistoryRepository`, `CareerMappingRepository`)

Fix: removed the `delete()` methods from `PsychProfilePersistenceAdapter`, `SessionHistoryPersistenceAdapter`, `CareerMappingPersistenceAdapter`. Also removed `findById(UUID)` and `save(TraitDimension)` from `TraitDimensionPersistenceAdapter` which were not in `TraitDimensionRepository`.

This document is complete and permanently frozen as of 2026-03-07.
psych-svc: 12/12 tests PASS — BUILD SUCCESS — 7.266 s